



15

CHAPTER

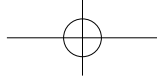
네트워크

Objectives

- TCP/IP 프로토콜의 개념을 이해한다.
- 자바의 소켓과 포트의 개념을 이해한다.
- 서버 클라이언트 통신 프로그램의 구조를 이해한다.
- 서버 소켓과 클라이언트 소켓을 구분하여 이해한다.
- 간단한 채팅 프로그램 예제를 통해 소켓 통신을 이해한다.
- 수식 계산 서버-클라이언트 통신 예제로 서버-클라이언트 통신을 이해한다.
- 소켓 프로그래밍을 할 수 있다.

주요용어

TCP/IP, IP 주소, 소켓, Socket 클래스, 서버 소켓, ServerSocket 클래스, accept(), close()



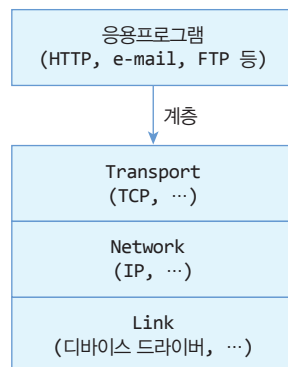
네트워크

15.1 TCP/IP

TCP/IP 프로토콜 소개

TCP 프로토콜
IP 프로토콜

TCP 프로토콜은 Transmission Control Protocol의 약자로 다른 두 시스템 간에 신뢰성 있는 데이터의 전송을 관장하는 통신 프로토콜로서 **IP(Internet Protocol) 프로토콜** 위에서 동작한다. TCP 프로토콜을 사용하는 응용프로그램으로는 e-mail, FTP, 웹(HTTP) 등이 있다. IP는 패킷 교환 네트워크에서 송신 호스트와 수신 호스트가 데이터를 주고받는 것을 관장하는 프로토콜로서 TCP의 하위 레벨 프로토콜이다. TCP는 IP 기능을 활용하여 두 시스템 사이에 데이터가 손상 없이 안전하게 전송되도록 하며, TCP와 IP를 묶어 TCP/IP로 표기한다. TCP/IP 프로토콜 및 e-mail, 웹 응용프로그램의 관계는 [그림 15-1]과 같다.



[그림 15-1] 네트워크 계층

IP 주소

IP 주소는 네트워크상에서 유일하게 식별될 수 있는 **네트워크 장치의 주소**로서, 예를 들면 192.156.11.15와 같이 4개의 숫자가 '.'으로 연결된다. 하나의 숫자 범위는 0~255로서 한 바이트로 표현이 가능하다. IP 주소는 마치 전화번호나 집주소와 같아 이 주소를 통해 네트워크에 연결된 장치를 식별할 수 있으며, 동일한 주소를 여러 네트워크 장치에 중복해서 사용할 수 없다. 숫자로 된 주소는 기억하기 어려우므로 www.naver.com과 같은 문자열로 구성된 도메인 이름으로 바꿔 사용한다. 사용자가 문자열로 구성된 도메인 이름을 사용하면 DNS(Domain Name System) 서버에 의해 숫자로 구성된 IP 주소로 자동 변환되게 된다.

현재는 4개의 숫자로 구성된 IP 주소를 표현하기 위해 32비트의 IP 버전 4(IPv4)가 사용되고 있다. 그러나 세계적으로 네트워크 장치의 개수가 폭발적으로 증가하여 각 장치에 고유하게 부여할 수 있는 IP 주소가 고갈됨에 따라 128비트의 IP 버전 6(IPv6)이 점점 사용되는 추세이다.

자신의 컴퓨터에서 자신의 IP 주소를 간단히 **localhost**라는 이름으로 사용해도 된다. localhost의 IP 주소는 127.0.0.1로 정해져 있다.

IP 주소

네트워크 장치의 주소

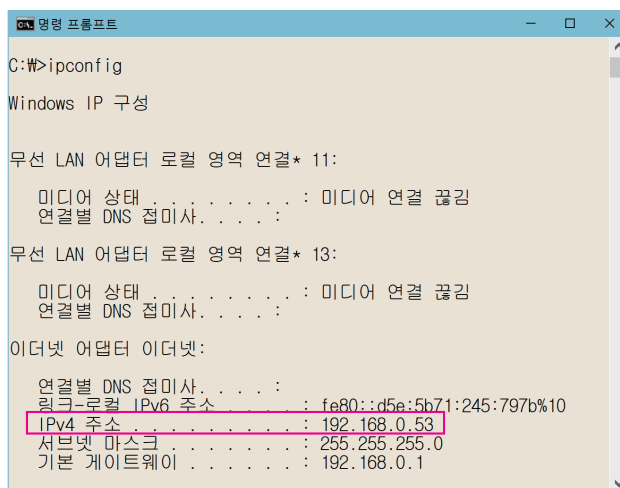
localhost

127.0.0.1

ipconfig

● 내 컴퓨터의 IP 주소 확인하기

윈도우 PC에서는 명령창을 열어 **ipconfig** 명령을 수행하면 [그림 15-2]와 같이 컴퓨터의 IP 주소를 확인할 수 있다. 대학이나 연구소 등에서는 한 컴퓨터에 항상 동일한 IP(고정 IP)를 설정하는 경우가 많지만, 가정에서는 대개 무선 공유기가 자동으로 할당해주는 IP 주소를 부여받는다. 이를 유동 IP라고 부른다.



[그림 15-2] 내 컴퓨터 IP 주소 확인

포트

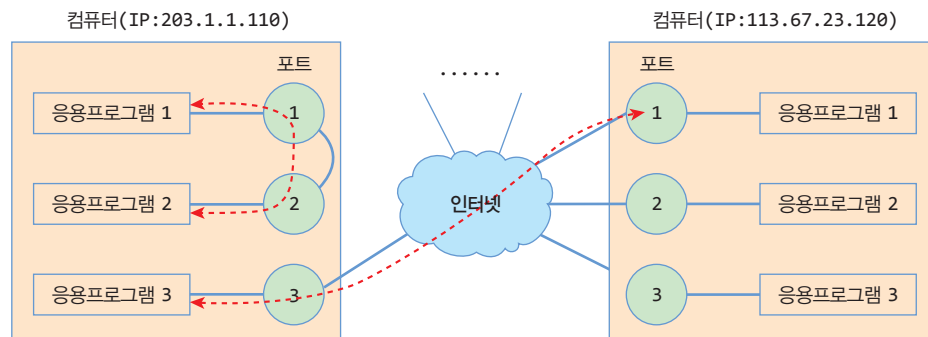


포트

IP 주소는 네트워크상에 있는 한 컴퓨터를 유일하게 식별한다. 하지만, 한 컴퓨터에는 여러 응용프로그램이 네트워크를 사용하고 있기 때문에, IP 주소만 가지고는 통신하고자 하는 응용프로그램을 식별할 수 없다. 이를 위해 한 컴퓨터 내의 각 응용프로그램은 통신을 위해 가상의 연결단인 **포트(port)**를 생성하고, 이 포트 번호로 상대방이 자신을 식별하게 한다.

IP 주소는 아파트의 동 번호와 같고, 포트 번호는 그 동에 있는 호 번호에 비유할 수 있다. 다른 예로 은행의 사례를 들면, IP 주소는 은행 지점의 주소이고, 포트 번호는 은행 내의 고객 창구 번호와 같다.

따라서 통신을 수행하는 모든 응용프로그램은 IP 주소와 포트를 이용하여 상대방 통신 프로그램을 인지하며 데이터를 교환한다. 물론 이때 상대방 응용프로그램은 자신의 IP 주소와 포트 번호를 알고 통신 접속이나 데이터가 오기를 기다리고 있어야 한다. 응용프로그램과 포트 사용 사례는 [그림 15-3]과 같다.



[그림 15-3] 포트를 이용한 두 응용프로그램의 통신

잘 알려진 포트

포트 번호는 응용프로그램 개발자가 임의로 선택하여 사용할 수 있으나, 기존 응용프로그램에서 사용하고 있는 포트 번호나 시스템의 포트 번호는 피하는 것이 좋다. 시스템이나 기존에 알려진 응용프로그램에서 사용하는 포트 번호를 **잘 알려진 포트** (well-known ports)라고 한다. 예를 들어, SSH는 22번 포트, HTTP는 80번 포트, FTP는 21번 포트 등이며, 이들은 주로 0~1023 사이의 번호를 가지므로 사용자가 작성하는 응용프로그램에서는 이 범위의 포트 번호는 피해서 선택하도록 한다.

- 1 IP 주소와 포트에 대해 설명하라.
- 2 현재 자신의 컴퓨터의 IP 주소가 얼마인지 확인하라.
- 3 한 컴퓨터에서 2개의 통신 응용프로그램이 동일한 포트 번호를 사용한다면 어떤 문제가 발생할 수 있는가?
- 4 하나의 통신 응용프로그램은 반드시 하나의 포트만 사용하여야 하는가?

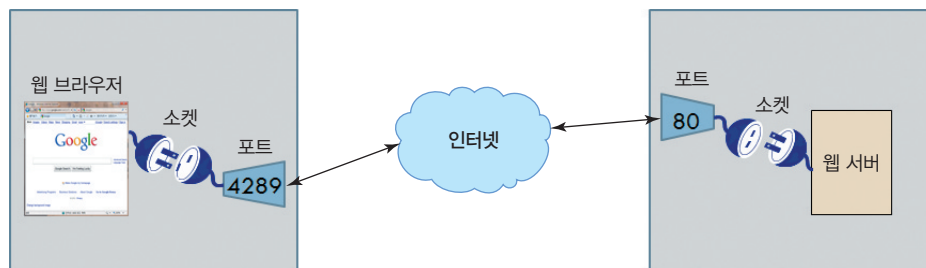


15.2 소켓 프로그래밍

소켓(socket)

소켓 통신은 개발자가 TCP/IP 네트워크를 이용하여 쉽게 통신 프로그램을 작성하도록 지원하는 기반 기술이다. 여기서 소켓은 통신하는 두 응용프로그램 간의 통신 링크의 각 끝단(endpoint)으로서, TCP/IP의 네트워크 기능을 활용하여 다른 컴퓨터의 소켓과 데이터를 주고받는다. 소켓을 활용하는 통신의 모양은 [그림 15-4]와 같으며, 소켓은 특정 포트에 연결되어 데이터를 보내거나 받을 때 해당 응용프로그램을 식별한다.

소켓 통신
소켓



[그림 15-4] 소켓을 이용하는 통신 사례

응용프로그램은 소켓과 연결한 후 소켓에 데이터를 주기만 하면, 소켓이 상대방 응용프로그램에 연결된 소켓에 데이터를 보낸다. 또는 응용프로그램은 연결된 소켓으로부터 도착한 데이터를 단순히 받기만 하면 된다. 인터넷을 경유하여 데이터를 주고받는 기능은 순전히 소켓의 몫이다. 데이터를 주고받는 동안 전송받은 데이터에 오류가 없는 지 검사하고, 만일 손상된 데이터가 오면 다시 받기를 요청하는 등 온전한 데이터를 받는 과정은 모두 소켓의 몫이다.

소켓과 서버 클라이언트 통신

서버
클라이언트

소켓을 이용하는 통신에서는 반드시 서버 응용프로그램과 클라이언트 응용프로그램으로 구분된다. 정보를 제공하는 쪽을 **서버(server)**라고 부르며, 정보를 이용하는 쪽을 **클라이언트(client)**라고 부른다. 통신은 서버가 먼저 클라이언트의 접속을 기다리고, 클라이언트에서 서버에 접속하면, 그 때부터 서버나 클라이언트가 데이터를 서로 주고받을 수 있다. 서버나 클라이언트가 보내는 순서를 정하거나 순서에 상관없이 데이터를 전송하는 것은 개발자가 프로그램을 작성하기에 달려 있다.

● 서버 소켓과 클라이언트 소켓

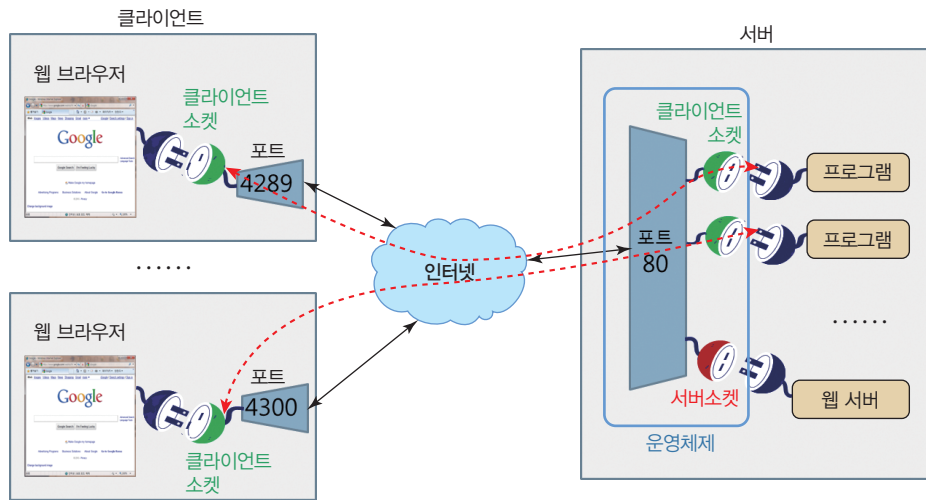
서버 소켓
클라이언트 소켓

소켓에는 **서버 소켓**과 **클라이언트 소켓**의 2가지 종류가 있다. 이 둘은 다음과 같이 용도가 서로 다르다.

- 서버 소켓은 서버 응용프로그램이 사용자의 접속을 기다리는(**listen**) 목적으로만 사용된다.
- 클라이언트 응용프로그램에서는 클라이언트 소켓을 이용하여 서버에 접속한다.
- 서버 소켓은 클라이언트가 접속해오면, 클라이언트 소켓을 추가로 만들어 상대 클라이언트와 통신하게 한다.

이 내용을 정리하면 서버 소켓은 클라이언트의 접속을 기다리는 소켓이며, 클라이언트 소켓은 데이터 통신을 실시하는 소켓이다.

[그림 15-5]는 소켓을 이용한 전형적인 웹 서버 클라이언트 통신 프로그램의 구조와 함께, 클라이언트 응용프로그램과 클라이언트 소켓, 그리고 서버 응용프로그램에서 서버 소켓, 클라이언트 소켓의 관계를 보여준다.



[그림 15-5] 소켓을 이용한 웹 서버와 웹 클라이언트 사이의 통신

● 서버에서 클라이언트 소켓들의 포트 공유

[그림 15-5]에서 서버 쪽의 통신프로그램은 각각 독립된 소켓을 이용하여 클라이언트와 통신을 수행한다. 한편, 그림의 서버 쪽을 자세히 들여다보면, '동일한 **포트(80)**를 여러 클라이언트 소켓들이 **공유**하고 있으면, 여러 클라이언트들로부터 전송받은 데이터를 서버 내 어떤 소켓으로 전달해야 하는지 어떻게 판단할까?' 하는 의문이 들 것이다. 이것은 운영체제에 의해 처리된다. 클라이언트가 처음 서버 소켓에 연결될 때, **운영체제**는 연결된 클라이언트 IP 주소와 포트 번호를 저장하고 기억해둔다. 그 후 서버 컴퓨터의 운영체제는 클라이언트로부터 데이터 패킷을 받게 되면, 패킷 속에 들어 있는 클라이언트의 IP 주소와 포트 번호를 참고하여, 서버에 있는 클라이언트 소켓을 찾아 그 곳으로 데이터를 보낸다.

포트 공유
운영체제

● 소켓을 이용한 서버 클라이언트 통신 프로그램 구성

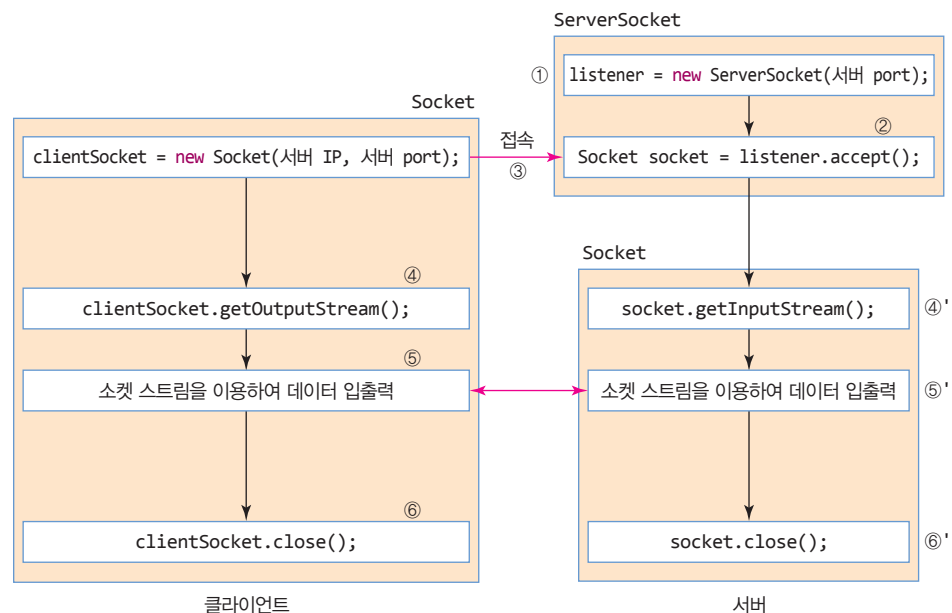
자바로 작성하는 서버 응용프로그램과 클라이언트 응용프로그램의 구조를 전체적으로 알아보자. 구체적인 코드들은 다음 절에서 설명한다. 서버 클라이언트의 전형적인 구조는 [그림 15-6]과 같으며 동작하는 과정은 다음과 같다.

1. 서버 응용프로그램은 `ServerSocket` 클래스를 이용하여 **서버 소켓 객체**를 생성하고(`new ServerSocket(서버 port)`①) 클라이언트의 **접속**을 받기 위해 **기다린다**(②). 서버 소켓을 생성할 때 포트 번호를 주어 해당 포트에 접속해 오는 클라이언트 기다리게 한다.
2. 클라이언트 응용프로그램은 `Socket` 클래스를 이용하여 클라이언트 소켓 객체를

서버 소켓 객체
접속 기다린다

서버에 접속
클라이언트 소켓을 따로 생성
소켓을 닫는다

- 생성하고(new Socket(서버 IP, 서버 port)) 서버에 접속을 시도한다(③). 소켓 객체를 생성할 때, 접속할 서버 소켓의 IP 주소와 포트 번호를 지정한다.
3. 서버는 클라이언트로부터 접속 요청을 받으면, accept() 메소드에서 접속된 클라이언트와 통신하도록 전용 클라이언트 소켓을 따로 생성한다.
 4. 서버와 클라이언트 모두 소켓으로부터 입출력 스트림을 얻어내고 데이터를 주고받을 준비를 한다(④, ④').
 5. 서버에 생성된 클라이언트 전용 소켓과 클라이언트의 소켓이 상호 연결된 채 스트림을 이용하여 양방향으로 데이터를 주고받는다(⑤, ⑤').
 6. 서버는 클라이언트가 접속해 올 때마다 accept() 메소드에서 따로 전용 클라이언트 소켓을 생성하여 클라이언트와 통신하도록 한다. 통신이 끝나면 소켓을 닫는다(⑥, ⑥').



[그림 15-6] 소켓을 이용한 서버 클라이언트 통신 프로그램의 전형적인 구조

Socket 클래스, 클라이언트 소켓

Socket
클라이언트 소켓

Socket은 java.net 패키지에 포함되어 있는 클래스로서 클라이언트 소켓을 구현한다. 즉, 서버와 통신하기 위해 클라이언트 응용프로그램에서 사용하는 소켓이다. Socket의 생성자와 메소드는 각각 <표 15-1>, <표 15-2>와 같다.

Socket의 생성자는 연결할 서버의 IP 주소(또는 도메인 주소)와 포트 번호를 인자로 받아서 Socket 객체를 생성한다. 클라이언트 자신의 주소와 포트 번호가 아님에 주

의하라. 이제 Socket 클래스를 이용하여 클라이언트 응용프로그램을 작성하는 방법을 하나씩 알아보자.

● 클라이언트 소켓 생성 및 서버 접속

IP 주소가 128.12.1.1이고 포트 번호가 5550인 서버에 연결하기 위해 다음과 같이 클라이언트 소켓 객체를 생성한다. 이때 클라이언트의 포트(local port)는 사용되지 않는 포트 중에서 자동으로 선택된다.

```
Socket clientSocket = new Socket("128.12.1.1", 5550); // 128.12.1.1 서버에 접속
```

Socket 객체가 생성되면 곧바로 128.12.1.1 주소의 5550번 포트로 자동 접속이 이루어진다. 다음과 같이 빈 소켓 객체를 생성하고 서버에 접속해도 된다.

```
Socket clientSocket = new Socket(); // 연결되지 않는 소켓 생성
clientSocket.bind(new InetSocketAddress("192.168.1.21", 1234));
// 소켓에 자신의 IP 주소(192.168.1.21)와 로컬 포트(1234)를 결합한다.
clientSocket.connect(new InetSocketAddress("128.12.1.1", 5550));
// IP 주소가 128.12.1.1이고 포트가 5550인 서버 응용프로그램에 접속
```

● 네트워크 입출력 스트림 생성

소켓이 만들어지고 서버와 연결이 된 후에는 Socket 클래스의 `getInputStream()`과 `getOutputStream()` 메소드를 이용하여 서버와 데이터를 주고받을 소켓 스트림을 얻어내고 이를 버퍼 스트림에 연결한다.

```
getInputStream()
getOutputStream()
```

```
BufferedReader in = new BufferedReader(new
    InputStreamReader(clientSocket.getInputStream()));
BufferedWriter out = new BufferedWriter(new
    OutputStreamWriter(clientSocket.getOutputStream()));
```

지금부터는 in, out 스트림 객체를 이용하여 네트워크 데이터를 보내고 받으면 된다. 이 코드에서 만들어진 in, out은 문자만 보내고 받을 수 있는 문자 입출력 스트림이다.

● 서버로 데이터 전송

이제 버퍼 출력 스트림 out을 통해 데이터를 전송해보자. 다음은 "hello" 문자열을 서버로 전송하는 코드이다.

```
out.write("hello"+"\\n");
out.flush();
```

out.flush()

"hello"에 "\\n"을 덧붙여 보내는 이유는 서버 쪽에서 라인 단위('\\n' 문자가 올 때까지 한 번에 읽는)로 수신한다고 가정하였기 때문이다. 스트림 out은 버퍼 입출력 스트림이므로 버퍼가 차기 전까지 데이터를 보내지 않기 때문에 강제로 out.flush()를 호출하여 스트림 속의 데이터를 모두 즉각 전송하도록 하였다.

● 서버로부터 데이터 수신

버퍼 입력 스트림 in을 이용하면 서버로부터 문자 데이터를 수신할 수 있다. 다음은 클라이언트로부터 한 개의 문자를 입력받는 코드이다.

```
int x = in.read(); // 클라이언트로부터 한 개의 문자 수신
```

한 행의 문자열을 입력받는 코드는 다음과 같다.

```
String line = in.readLine(); // 클라이언트로부터 한 행의 문자열 수신
```

in.readLine() 메소드는 '\\n' 문자가 올 때까지 계속 읽고 '\\n'이 도착하면 그때까지 읽은 문자열을 리턴한다. 이 문자열 속에는 '\\n'이 삽입되지 않는다.

● 데이터 송수신 종료

데이터 송수신을 모두 수행하고 소켓 연결을 끊고자 하면 다음과 같이 한다.

```
socket.close();
```

〈표 15-1〉

Socket 클래스의 생성자

생성자	설명
Socket	연결되지 않은 상태의 소켓을 생성
Socket(InetAddress address, int port)	소켓을 생성하고, 지정된 IP 주소(addresss)와 포트 번호(port)에서 대기하는 원격 응용프로그램의 소켓에 연결
Socket(String host, int port)	소켓을 생성하여 지정된 호스트(host)와 포트 번호(port)에 연결한다. 호스트 이름이 null인 경우는 루프백(loopback) 주소로 가정

메소드	설명
<code>void bind(SocketAddress bindpoint)</code>	소켓에 로컬 IP 주소와 로컬 포트 지정(결합)
<code>void close()</code>	소켓을 닫는다.
<code>void connect(SocketAddress endpoint)</code>	서버에 연결
<code>InetAddress getInetAddress()</code>	소켓에 연결된 서버 IP 주소 반환
<code>InputStream getInputStream()</code>	소켓의 입력 스트림 반환. 이 스트림을 이용하여 소켓이 상대방으로부터 받은 데이터를 읽을 수 있음
<code>InetAddress getLocalAddress()</code>	소켓의 로컬 주소 반환
<code>int getLocalPort()</code>	소켓의 로컬 포트 번호 반환
<code>int getPort()</code>	소켓에 연결된 서버의 포트 번호 반환
<code>OutputStream getOutputStream()</code>	소켓의 출력 스트림 반환. 이 스트림에 출력하면 소켓이 서버로 데이터 전송
<code>boolean isBound()</code>	소켓이 로컬 주소와 결합되어 있으면 true 반환
<code>boolean isConnected()</code>	소켓이 서버에 연결되어 있으면 true 반환
<code>boolean isClosed()</code>	소켓이 닫혀있으면 true 반환
<code>void setSoTimeout(int timeout)</code>	데이터 읽기 타임아웃 시간 지정. 0이면 타임아웃 해제

〈표 15-2〉

Socket 클래스의 주요 메소드

ServerSocket 클래스, 서버 소켓

ServerSocket 클래스는 **서버 소켓**을 구현한다. **ServerSocket** 클래스는 **java.net** 패키지에 포함되어 있으며 **ServerSocket** 클래스의 생성자와 메소드는 각각 〈표 15-3〉, 〈표 15-4〉와 같다. **ServerSocket**은 클라이언트로부터 **연결 요청을 기다리는 목적**으로만 사용되며, 서버가 클라이언트의 연결 요청을 수락하면 **Socket** 객체를 별도로 생성하고, 이 **Socket** 객체가 클라이언트와 데이터를 주고받는다. **ServerSocket**은 데이터의 송수신에 사용되지 않는다.

ServerSocket
서버 소켓
연결 요청을 기다리는 목적

● 서버 소켓 생성

ServerSocket 생성자는 포트 번호를 인자로 받아서 **ServerSocket** 객체를 생성한다. 이 포트 번호는 클라이언트의 접속을 기다릴 자신의 포트 번호이다. 이미 사용 중인 포트 번호를 지정하면 오류가 발생한다. 9999번 포트를 사용하는 서버 소켓을 생성하는 예를 들면 다음과 같다.

```
ServerSocket listener = new ServerSocket(9999);
```

accept()

● 클라이언트로부터 접속 대기

ServerSocket 클래스의 `accept()` 메소드를 이용하여 클라이언트로부터의 연결 요청을 기다린다. `accept()` 메소드가 연결을 수락하면 다음과 같이 Socket 객체를 하나 별도로 생성하여 리턴한다.

```
Socket socket = listener.accept();
```

서버에서 지금 접속된 클라이언트와의 데이터 통신은 새로 만들어진 `socket`을 이용하여 이루어진다. 새로 만들어진 `socket`은 서버 소켓과 동일하게 9999번 포트를 통해 데이터를 주고받는다.

getInputStream()
getOutputStream()

● 네트워크 입출력 스트림 생성

클라이언트로 데이터를 주고 받기 위한 스트림 객체는, ServerSocket의 `accept()` 메소드로부터 얻은 `socket` 객체의 `getInputStream()`과 `getOutputStream()` 메소드를 이용하여 얻어낸다. 다음과 같이 소켓 스트림을 버퍼 입출력 스트림에 연결하여 사용한다.

```
BufferedReader in = new BufferedReader(new
    InputStreamReader(socket.getInputStream()));
BufferedWriter out = new BufferedWriter(new
    OutputStreamWriter(socket.getOutputStream()));
```

버퍼 입출력 스트림 `in`, `out`을 이용하여 클라이언트와 데이터를 주고받으면 된다. `in`, `out`은 모두 문자만 입출력하는 스트림이다.

● 클라이언트로부터 데이터 수신

앞서 만들어진 버퍼 스트림 `in`을 이용하여 클라이언트로부터 문자 데이터를 수신할 수 있다. 다음은 클라이언트로부터 한 개의 문자를 입력받는 코드이다.

```
int x = in.read(); // 클라이언트로부터 한 개의 문자 수신
```

한 행의 문자열을 입력받는 코드는 다음과 같다.

```
String line = in.readLine(); // 클라이언트로부터 한 행의 문자열 수신
```

`in.readLine()` 메소드는 '\n' 문자가 올 때까지 계속 읽고 '\n'이 도착하면 그때까지 읽은 문자열을 리턴한다. 이 문자열 속에는 '\n'이 삽입되지 않는다. 현재 서버에서 라인 단위로 읽기 때문에 클라이언트에서는 송신하는 데이터의 끝에 '\n'을 덧붙여 보내야 한다.

● 클라이언트로 데이터 전송

앞서 만들어진 문자 버퍼 스트림 `out`을 통해 클라이언트로 데이터를 전송할 수 있다. 다음은 "Hi!, Client" 문자열을 클라이언트로 전송하는 코드이다.

```
out.write("Hi!, Client"+"\\n");
out.flush();
```

"Hi!, Client"에 "\\n"을 덧붙여 보내는 이유는 클라이언트 쪽에서 라인 단위('\n' 문자가 올 때까지 한 번에 읽는)로 수신한다고 가정하였기 때문이다. `out.flush()`를 호출하면 버퍼 스트림 속의 데이터를 모두 즉각 클라이언트로 전송한다.

```
out.flush()
```

● 데이터 송수신 종료

데이터 송수신을 모두 수행하고 소켓 연결을 끊고자 하면 다음과 같이 한다.

```
socket.close();
```

● 서버 응용프로그램 종료

더 이상 클라이언트의 접속을 받지 않고 서버 응용프로그램을 종료하고자 하는 경우 다음과 같이 `ServerSocket`을 종료시킨다.

```
serverSocket.close();
```

생성자	설명
<code>ServerSocket(int port)</code>	지정된 포트 번호(port)와 결합된 소켓 생성

〈표 15-3〉

`ServerSocket` 클래스의 주요 생성자

〈표 15-4〉

ServerSocket 클래스의 주요
메소드

메소드	설명
Socket accept()	클라이언트로부터 연결 요청을 기다리다 요청이 들어오면 수락하고 클라이언트와 데이터를 주고받을 새 Socket 객체를 반환
void close()	서버 소켓을 닫는다.
InetAddress getInetAddress()	서버 소켓의 로컬 IP 주소 반환
int getLocalPort()	서버 소켓의 로컬 포트 번호 반환
boolean isBound()	서버 소켓이 로컬 주소와 결합되어 있으면 true 반환
boolean isClosed()	서버 소켓이 닫혀있으면 true 반환
void setSoTimeout(int timeout)	accept()가 대기하는 타임아웃 시간 지정. 0이면 무한정 대기

15.3 서버-클라이언트 채팅 프로그램 만들기

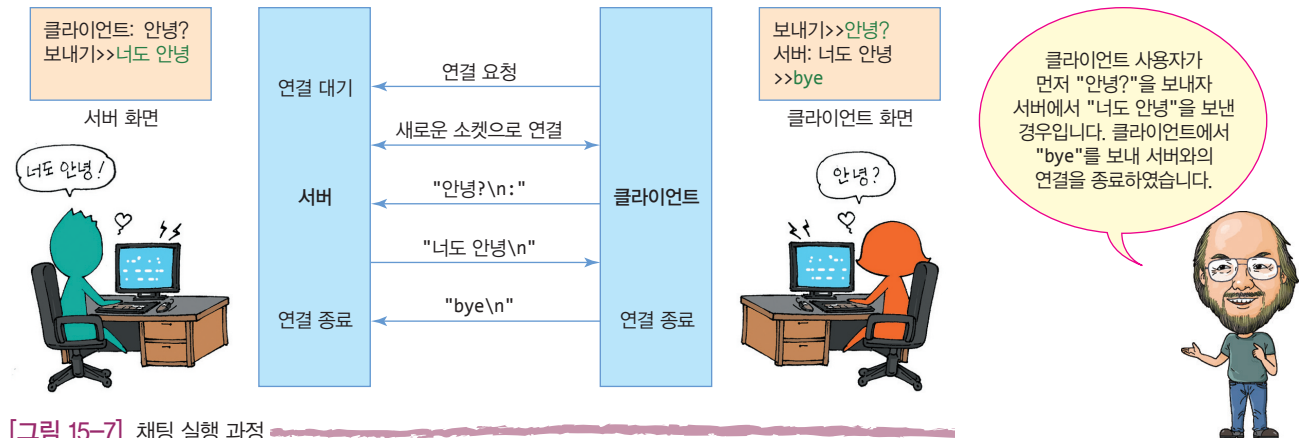
예제 개요

채팅 예제

서버와 클라이언트가 번갈아 한 번씩 채팅 문자를 보내는 간단한 채팅 예제를 만들어 보자. 채팅 프로그램의 개념은 다음과 같다.

- 서버와 클라이언트가 1:1로 채팅한다.
- 클라이언트와 서버가 서로 한번씩 번갈아 가면서 문자열 전송 및 수신한다. 클라이언트가 먼저 문자열을 보내면, 서버가 받아 출력하고 서버가 다시 문자열을 보내는 식이다.
- 서버나 클라이언트는 사용자로부터 문자열을 입력받아 보낸다. 이때 문자열 끝에 "\n"을 덧붙여 보내고 받는 쪽에서는 라인 단위로 수신한다.
- 클라이언트가 "bye"를 보내면 서버 클라이언트 모두 종료한다.

간단한 채팅 프로그램 예제를 통해 자바에서 소켓을 이용하여 어떻게 클라이언트와 서버 간에 데이터를 주고받는 지 실습해보기로 한다. 채팅 프로그램의 실행 과정은 [그림 15-7](#)과 같다.



[그림 15-7] 채팅 실행 과정

코드 부분 설명

채팅 프로그램의 전체 소스 코드는 다음 절에서 보이며, 여기서는 소스 코드 중 두 부분만 간단히 설명한다. 나머지는 15.2절에서 설명한 바와 거의 같다.

● 클라이언트의 소켓 생성

서버와 클라이언트를 동일한 컴퓨터에서 실행해보고자 한다. 그러므로 클라이언트에서 소켓을 생성할 때, 다음과 같이 서버의 IP 주소 부분을 "localhost"로 하면 된다. 서버가 다른 컴퓨터라면 서버의 IP 주소를 주어야 한다.

```
socket = new Socket("localhost", 9999); // 클라이언트 소켓 생성. 서버의 9999번 포트에 연결
```

● 사용자로부터 라인 입력

다음 절의 소스 코드에는 15.2절에서 설명한 내용에 클라이언트나 서버에서 사용자가 입력한 문자열을 읽어 출력하는 부분이 추가되어 있다. 다음은 서버 쪽에서 사용자가 입력한 문자열을 전송하는 부분 코드이다.

```
Scanner scanner = new Scanner(System.in);
String outputMessage = scanner.nextLine(); // 키보드로부터 한 줄을 읽는다.
out.write(outputMessage + "\n");
out.flush();
```

서버-클라이언트 채팅 소스 코드

전체 예제 코드는 다음과 같다.

• 서버 프로그램 ServerEx.java

```

1  import java.io.*;
2  import java.net.*;
3  import java.util.*;
4
5  public class ServerEx {
6      public static void main(String[] args) {
7          BufferedReader in = null;
8          BufferedWriter out = null;
9          ServerSocket listener = null;
10         Socket socket = null;
11         Scanner scanner = new Scanner(System.in); // 키보드에서 읽을 scanner 객체 생성
12         try {
13             listener = new ServerSocket(9999); // 서버 소켓 생성
14             System.out.println("연결을 기다리고 있습니다....");
15             socket = listener.accept(); // 클라이언트로부터 연결 요청 대기
16             System.out.println("연결되었습니다.");
17             in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
18             out = new BufferedWriter(new OutputStreamWriter(socket.getOutputStream()));
19             while (true) {
20                 String inputMessage = in.readLine(); // 클라이언트로부터 한 행 읽기
21                 if (inputMessage.equalsIgnoreCase("bye")) {
22                     System.out.println("클라이언트에서 bye로 연결을 종료하였음");
23                     break; // "bye"를 받으면 연결 종료
24                 }
25                 System.out.println("클라이언트: " + inputMessage);
26                 System.out.print("보내기>>"); // 프롬프트
27                 String outputMessage = scanner.nextLine(); // 키보드에서 한 행 읽기
28                 out.write(outputMessage + "\n"); // 키보드에서 읽은 문자열 전송
29                 out.flush(); // out의 스트림 버퍼에 있는 모든 문자열 전송
30             }
31         } catch (IOException e) {
32             System.out.println(e.getMessage());
33         } finally {
34             try {
35                 scanner.close(); // scanner 닫기
36                 socket.close(); // 통신용 소켓 닫기
37                 listener.close(); // 서버 소켓 닫기
38             } catch (IOException e) {
39                 System.out.println("클라이언트와 채팅 중 오류가 발생했습니다.");
40             }
41         }
42     }
43 }

```

소켓 입력 스트림

소켓 출력 스트림

받은 메시지를 화면에 출력

• 클라이언트 프로그램 ClientEx.java

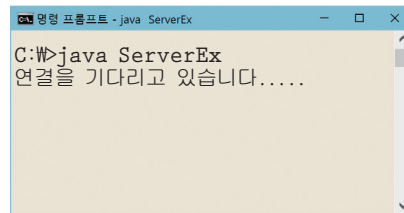
```
1  import java.io.*;
2  import java.net.*;
3  import java.util.*;
4
5  public class ClientEx {
6      public static void main(String[] args) {
7          BufferedReader in = null;
8          BufferedWriter out = null;
9          Socket socket = null;
10         Scanner scanner = new Scanner(System.in); // 키보드에서 읽을 scanner 객체 생성
11         try {
12             socket = new Socket("localhost", 9999); // 클라이언트 소켓 생성. 서버에 연결
13             in = new BufferedReader(new InputStreamReader(socket.getInputStream())); // 소켓 입력 스트림
14             out = new BufferedWriter(new OutputStreamWriter(socket.getOutputStream())); // 소켓 출력 스트림
15             while (true) {
16                 System.out.print("보내기>>"); // 프롬프트
17                 String outputMessage = scanner.nextLine(); // 키보드에서 한 행 읽기
18                 if (outputMessage.equalsIgnoreCase("bye")) {
19                     out.write(outputMessage+"\n"); // "bye" 문자열 전송
20                     out.flush();
21                     break; // 사용자가 "bye"를 입력한 경우 서버로 전송 후 실행 종료
22                 }
23                 out.write(outputMessage + "\n"); // 키보드에서 읽은 문자열 전송
24                 out.flush(); // out의 스트림 버퍼에 있는 모든 문자열 전송
25                 String inputMessage = in.readLine(); // 서버로부터 한 행 수신
26                 System.out.println("서버: " + inputMessage); // 서버로부터 받은 메시지를 화면에 출력
27             }
28         } catch (IOException e) {
29             System.out.println(e.getMessage());
30         } finally {
31             try {
32                 scanner.close();
33                 if(socket != null) socket.close(); // 클라이언트 소켓 닫기
34             } catch (IOException e) {
35                 System.out.println("서버와 채팅 중 오류가 발생했습니다.");
36             }
37         }
38     }
39 }
```

서버-클라이언트 채팅 예제 실행

예제를 실행하기 전에 컴파일된 클래스 파일 `ServerEx.class`와 `ClientEx.class`를 드라이브 `c:\` 밑에 옮겨 놓아야 한다. 서버와 클라이언트는 따로 명령창을 열어 실행한다. 실행은 다음 순서로 진행한다.

연결 요청을 기다린다

1. 명령창을 열고 서버 프로그램을 먼저 실행시켜 클라이언트의 **연결 요청을 기다린다**. [그림 15-8]과 같다.



[그림 15-8] 서버가 먼저 실행되어 클라이언트로부터 연결을 기다리고 있는 서버 명령창

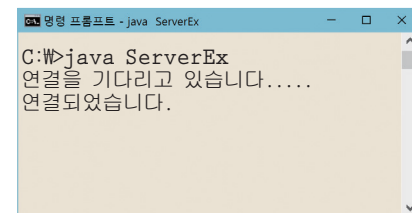
서버와 연결

2. [그림 15-9](a)와 같이 다른 명령창을 열어 클라이언트 프로그램을 실행한다. 클라이언트 프로그램이 실행되면 바로 서버로 연결을 시도한다. **서버와 연결**이 이루어지면 [그림 15-19](b)와 같이 서버의 명령창에는 "연결되었습니다."가 출력된다.



(a) 클라이언트 명령창

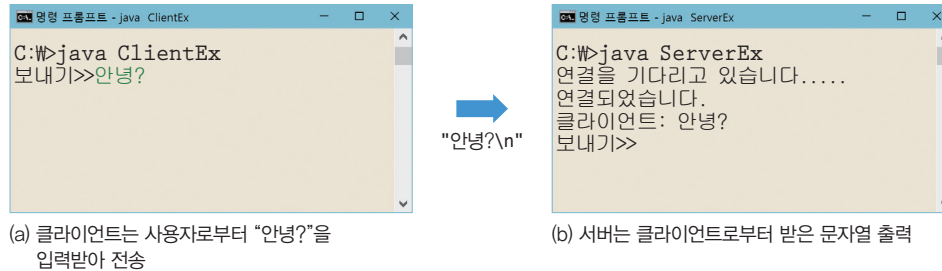
연결



(b) 서버의 명령창

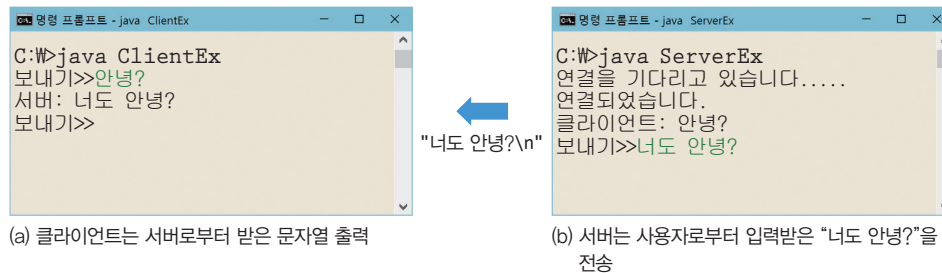
[그림 15-9] 클라이언트에서 서버로 문자열 보내기

3. 채팅은 클라이언트에서 먼저 시작하며, 번갈아 메시지를 주고받는다. [그림 15-10] (a)에서 클라이언트는 사용자로부터 "안녕?"과 <Enter> 키가 입력되면 서버로 전송한다. [그림 15-10](b)에서 서버는 받은 문자열을 출력한다.



[그림 15-10] 클라이언트에서 서버로 문자열 보내기

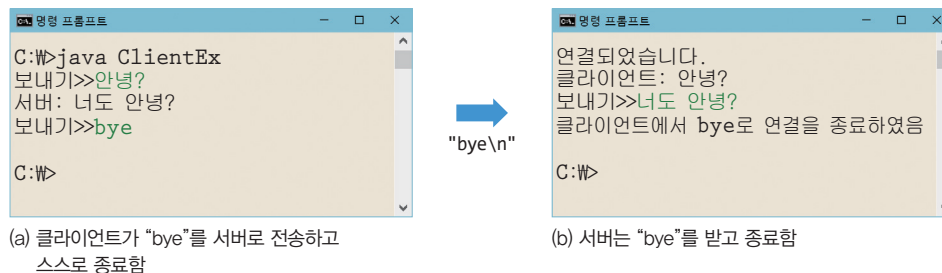
4. 이제, 서버는 사용자로부터 "너도 안녕?"을 입력받아 클라이언트로 보낸다. [그림 15-11](b)에서 서버는 사용자로부터 입력받은 "너도 안녕?"을 보낸다. [그림 15-11](a)에는 클라이언트가 서버로부터 받은 문자열을 출력한다. 이런 식으로 번갈아 메시지를 주고받는다.



[그림 15-11] 서버에서 클라이언트로 문자열 보내기

5. 클라이언트에서 "bye"를 서버에게 전송하면 채팅이 종료된다. [그림 15-12](a)는 클라이언트에서 "bye"를 입력받아 서버로 보내고 스스로 종료한 화면이고, [그림 15-12](b)는 "bye" 문자열을 받고 서버가 종료하는 화면이다.

채팅 종료



[그림 15-12] 클라이언트가 "bye"를 전송하면 채팅 종료



잠깐!

윈도우 운영체제에서 소켓을 이용하는 자바 응용프로그램을 실행하다가 비정상적으로 종료한 경우, 다시 실행시키면 다음 오류 메시지가 출력되고 실행이 되지 않는 경우가 더러 발생한다.

```
java.net.BindException: Address already in use: JVM_Bind
```

이것은 소켓이 포트를 바인드(소유)하여 사용하다가 바인드를 풀어 놓지 않고 (비정상)종료된 상황에서, 그 포트를 바인드할 때 나타나는 오류이다. 이런 경우, 명령창에 다음 명령을 입력하면 현재 윈도우에서 사용되고 있는 TCP 포트와 포트를 소유하고 있는 프로세스의 번호(PID) 리스트를 출력한다.

```
c:\> netstat -a -n -o
```

명령창에 출력된 리스트에서 문제가 된 포트와 이를 소유하고 있는 프로세스 번호(PID)를 찾아 프로세스를 죽이면 된다. 만일 프로세스 번호가 2692라고 하면 명령창에 다음 명령을 입력한다.

```
c:\> taskkill /f /pid 2692
```

이제 통신 프로그램을 실행하면 정상적으로 실행될 것이다.

15.4 수식 계산 서버-클라이언트 만들기 실습

문제 개요

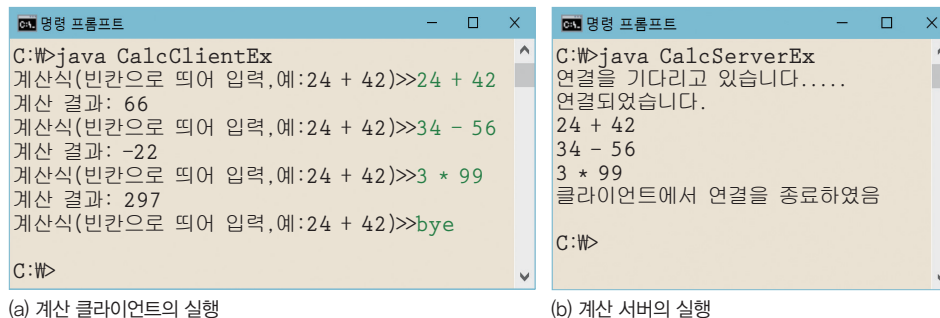
독자들의 실습 시간이다. 클라이언트가 수식을 보내면 서버가 수식을 계산한 후 답을 전송하는 통신 프로그램을 만들어 보자. 프로그램의 구체적인 내용은 다음과 같다.

- 서버 클라이언트는 1:1 통신한다.
- 서버를 먼저 실행시키고 클라이언트를 실행시켜 서버에 접속한다.
- 클라이언트는 사용자로부터 수식을 입력받아 서버로 전송한다. 연산자는 +, -, * 의 3가지만 허용하고 정수 연산만 가능하다.

- 서버가 식을 받으면 식을 서버의 화면에 출력하고, 계산하여 결과를 클라이언트로 전송한다.
- 클라이언트는 서버로부터 받은 답을 화면에 출력한다.
- 클라이언트와 서버는 전송할 데이터를 문자열로 만들고 "\n"을 덧붙여 전송하며, 받는 쪽에서는 라인 단위로 수신한다.
- 클라이언트가 "bye"를 보내면 양쪽 모두 종료한다.

실행 예시

[그림 15-13]은 수식 계산 통신 프로그램의 실행 과정 예시를 보여준다. 실행 전에 CalcClientEx.class와 CalcServerEx.class 파일은 미리 c:\에 옮겨져 있어야 한다. 서버가 먼저 실행되어 기다리며 클라이언트가 실행되면 연결이 이루어진다. 클라이언트가 사용자로부터 24 + 32의 수식을 입력받아 서버로 전송하고, 서버는 식을 계산하여 결과를 전송한다. 클라이언트는 결과 값을 받아 출력하고 다시 사용자로부터 수식을 입력받는다. 이 과정이 반복된다. 24 + 32에서 숫자와 + 기호 사이에는 반드시 빈칸으로 뺀다.



[그림 15-13] 수식 계산 서버-클라이언트의 실행 과정

코드 부분 설명

이 실습 예제는 앞의 채팅 프로그램과 거의 유사하여 작성하는데는 큰 어려움이 없을 것으로 본다. 다만 서버 쪽에서는 전달받은 수식을 계산하는 calc() 메소드가 추가 작성되었는데 이 메소드에 대해서만 알아보자. 클라이언트부터 넘어오는 수식은 다음과 같은 모양의 문자열이다.

"24 + 32" 또는 "36 - 5", "5 * 6"

숫자와 연산기호 사이에 빈칸(' ')을 삽입하여 전송하도록 제한을 가한 이유는 StringTokenizer 클래스를 이용하여 숫자와 연산기호를 분리하기 쉽게 함이다(예제가 쉽도록). 예를 들어 exp에 "24 + 32"의 문자열이 들어 있다면, 다음 코드를 이용하여 빈칸(' ')을 기준으로 exp의 문자열을 분리하면,

```
StringTokenizer st = new StringTokenizer(exp, " ");
```

다음과 같이 3개의 토큰으로 분리된다.

```
토큰 0 : "24"
토큰 1 : "+"
토큰 2 : "32"
```

calc() 메소드는 토큰 1의 연산기호(+)에 따라 계산을 수행하고 결과를 문자열로 만들어 리턴한다.

수식 계산 서버-클라이언트 소스 코드

전체 예제 코드는 다음과 같다.

• 서버 프로그램 CalcServerEx.java

```
1  import java.io.*;
2  import java.net.*;
3  import java.util.*;
4
5  public class CalcServerEx {
6      public static String calc(String exp) {
7          StringTokenizer st = new StringTokenizer(exp, " ");
8          if (st.countTokens() != 3) return "error";
9
10         String res="";
11         int op1 = Integer.parseInt(st.nextToken());
12         String opcode = st.nextToken();
13         int op2 = Integer.parseInt(st.nextToken());
14         switch (opcode) {
15             case "+": res = Integer.toString(op1 + op2);
16                 break;
17             case "-": res = Integer.toString(op1 - op2);
18                 break;
19             case "*": res = Integer.toString(op1 * op2);
```

```
20         break;
21         default : res = "error";
22     }
23     return res;
24 }
25
26 public static void main(String[] args) {
27     BufferedReader in = null;
28     BufferedWriter out = null;
29     ServerSocket listener = null;
30     Socket socket = null;
31     try {
32         listener = new ServerSocket(9999); // 서버 소켓 생성
33         System.out.println("연결을 기다리고 있습니다....");
34         socket = listener.accept(); // 클라이언트로부터 연결 요청 대기
35         System.out.println("연결되었습니다.");
36         in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
37                                     // 소켓 입력스트림
38         out = new BufferedWriter(new OutputStreamWriter(socket.getOutputStream()));
39                                     // 소켓 출력 스트림
40
41         while (true) {
42             String inputMessage = in.readLine(); // 클라이언트로부터 한 행 읽기. 수식
43             if (inputMessage.equalsIgnoreCase("bye")) {
44                 System.out.println("클라이언트에서 연결을 종료하였음");
45                                     // 받은 메시지를 화면에 출력
46                 break; // "bye"를 받으면 연결 종료
47             }
48             System.out.println(inputMessage); // 받은 메시지를 화면에 출력
49             String res = calc(inputMessage); // 계산. 계산 결과는 res
50             out.write(res + "\n"); // 계산 결과 문자열 전송
51             out.flush();
52         }
53     } catch (IOException e) {
54         System.out.println(e.getMessage());
55     } finally {
56         try {
57             if(socket != null) socket.close(); // 통신용 소켓 닫기
58             if(listener != null) listener.close(); // 서버 소켓 닫기
59         } catch (IOException e) {
60             System.out.println("클라이언트와 채팅 중 오류가 발생했습니다.");
61         }
62     }
63 }
```

• 클라이언트 프로그램 CalcClientEx.java

```

1  import java.io.*;
2  import java.net.*;
3  import java.util.*;
4
5  public class CalcClientEx {
6      public static void main(String[] args) {
7          BufferedReader in = null;
8          BufferedWriter out = null;
9          Socket socket = null;
10         Scanner scanner = new Scanner(System.in); // 키보드에서 읽을 scanner 객체 생성
11         try {
12             socket = new Socket("localhost", 9999);
13                                     // 클라이언트 소켓 생성. 서버와 바로 연결
14             in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
15                                     // 소켓 입력 스트림
16             out = new BufferedWriter(new OutputStreamWriter(socket.getOutputStream()));
17                                     // 소켓 출력 스트림
18
19             while (true) {
20                 System.out.print("계산식(빈칸으로 띄어 입력,예:24 + 42)>>"); // 프롬프트
21                 String outputMessage = scanner.nextLine(); // 키보드에서 수식 읽기
22                 if (outputMessage.equalsIgnoreCase("bye")) {
23                     out.write(outputMessage+"\n"); // "bye" 문자열 전송
24                     out.flush();
25                     break; // 사용자가 "bye"를 입력한 경우 서버로 전송 후 연결 종료
26                 }
27                 out.write(outputMessage + "\n"); // 키보드에서 읽은 수식 문자열 전송
28                 out.flush();
29                 String inputMessage = in.readLine(); // 서버로부터 계산 결과 수신
30                 System.out.println("계산 결과: " + inputMessage);
31                                     // 서버로부터 받은 계산 결과 화면 출력
32             }
33         } catch (IOException e) {
34             System.out.println(e.getMessage());
35         } finally {
36             try {
37                 scanner.close();
38                 if(socket != null) socket.close(); // 클라이언트 소켓 닫기
39             } catch (IOException e) {
40                 System.out.println("서버와 채팅 중 오류가 발생했습니다.");
41             }
42         }
43     }
44 }

```

요약



SUMMARY

- TCP는 다른 두 시스템 간에 신뢰성 있는 데이터의 전송을 관장하는 프로토콜이며 IP는 패킷 교환 네트워크에서 송신 호스트와 수신 호스트가 데이터를 주고받는 것을 관장하는 프로토콜이다.
- IP 주소는 231.1.127과 같이 네트워크상에서 유일하게 식별될 수 있는 4개의 숫자로 구성된다.
- TCP 포트(port)는 프로그램 간에 TCP 통신을 위한 가상의 연결 단으로서 통신할 프로그램을 식별한다.
- 소켓은 네트워크상에서 통신하는 두 프로그램 간의 양방향 통신 링크의 한쪽 끝단을 의미하며 포트에 연결된다.
- 소켓 통신은 통신 프로그램의 기반 기술로서, 소켓 통신 라이브러리를 이용하여 쉽게 통신 프로그램을 개발할 수 있다.
- 소켓 통신은 서버와 클라이언트 모형으로 통신하는 기반 기술이므로, 어느 하나는 서버가 되고 나머지 하나는 클라이언트가 된다.
- 자바 소켓은 2종류로서 서버 소켓인 **ServerSocket** 클래스와 클라이언트 소켓인 **Socket** 클래스가 있다.
- 소켓 통신에서 클라이언트로 **Socket** 객체를 생성하고, **Socket** 객체에게 서버의 IP 주소(혹은 도메인 주소)와 포트 번호를 주어 서버에 접속하도록 한다.
- 소켓 통신에서 서버는 서버 소켓인 **ServerSocket** 객체를 생성하고 특정 포트와 결합시키고, **accept()** 메소드를 호출하여 클라이언트의 연결 요청을 기다리도록 한다.
- **ServerSocket** 객체는 클라이언트의 요청이 들어오면 연결을 완료하고, 데이터 통신을 이어갈 **Socket** 객체를 생성하여 반환한다. 그리고 다시 다른 클라이언트의 연결을 기다린다. 클라이언트와의 데이터 통신은 모두 이 **Socket** 객체에 의해 이루어진다. 그러므로 **ServerSocket**은 연결을 기다리는(**listen**) 역할만 하고, 연결된 후의 데이터의 통신은 **Socket** 클래스의 역할이다.
- **Socket** 클래스는 응용프로그램이 네트워크에 데이터를 전송하거나 수신을 쉽게 하도록, 입출력 스트림을 제공한다. 응용프로그램은 **Socket** 객체로부터 스트림을 알아내고 이 스트림에 입출력함으로써 네트워크로 데이터를 송수신할 수 있다.
- 연결을 종료할 때는 **Socket** 클래스의 **close()** 메소드를 호출한다.
- 서버 소켓을 닫으면 클라이언트는 더 이상의 연결 요청을 할 수 없다.



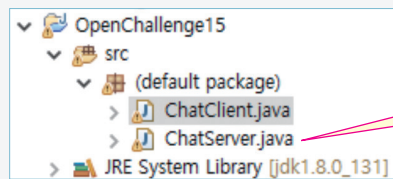
스레드를 이용한 서버-클라이언트 채팅

Open Challenge

목 적

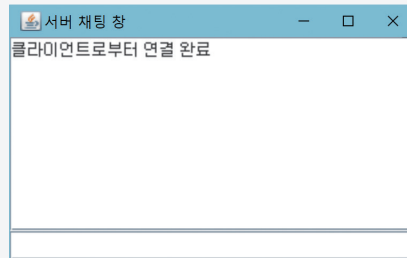
소켓 통신, 스레드를 이용하여
양방향 통신 작성

본문에서 다루었던 서버-클라이언트 1:1 채팅 예제는 클라이언트에서부터 시작하여 클라이언트와 서버가 순서대로 번갈아 가면서 메시지를 주고받았다. 이 예제를 순서에 상관없이 자유롭게 서버와 클라이언트가 메시지를 주고받을 수 있도록 스레드를 이용하여 스윙 프로그램으로 재작성하라. 서버와 클라이언트는 JTextArea를 이용하여 상대방으로부터 받은 메시지를 출력하고, JTextField 창을 이용하여 상대방에게 보낼 메시지를 사용자로부터 입력받고 <Enter>키를 입력하면 상대방에게 바로 전송하도록 하라. 본문의 예제와 달리 어느 한쪽이 접속을 끊으면 프로그램이 종료되도록 한다. 난이도 7

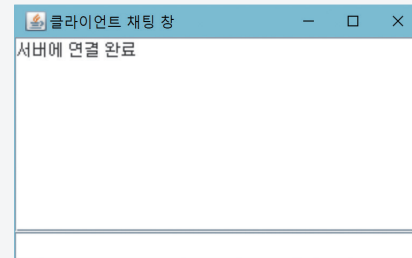


프로젝트 구성

CharServer.java를 먼저
실행한 후에 ChatClient.java를
실행한다.

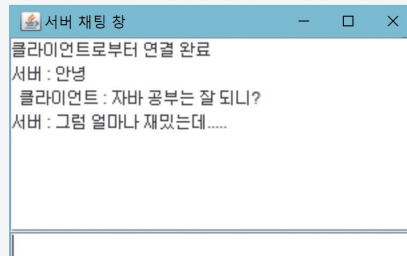


서버를 먼저 실행하고, 클라이언트 접속 기다림

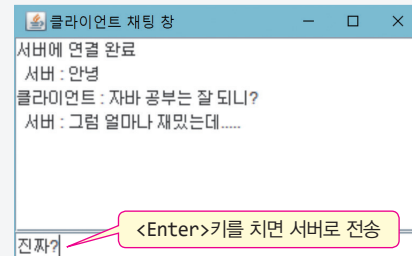


클라이언트가 실행되면 바로 서버에 접속함

접속



서버, 클라이언트가 순서에 상관없이 메시지를
주고 받는 화면



JTextField 창에 서버로 보낼 메시지를
입력하는 화면

메시지
전송

연습문제

EXERCISE



이론문제

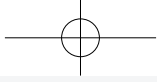
• 홀수 문제는 정답이 공개됩니다.

1. 윈도우 PC의 명령창에서 내 PC의 IP 주소를 확인하는 명령은 무엇인가? 독자들인 명령창에서 그 명령을 입력하여 독자 PC의 IP 주소를 확인해보라.
2. 잘 알려진 포트(well known port)에 대해 잘못 설명한 것은?
 - ① 기존의 많이 알려진 통신프로그램들이 전용으로 사용하는 포트를 일컫는다.
 - ② 잘 알려진 포트의 번호는 주로 0~1023 사이에 있다.
 - ③ SSH 응용프로그램은 22번 포트를 사용한다.
 - ④ 개발자는 응용프로그램에서 잘 알려진 포트를 사용해야 자신의 프로그램을 알리는 데 도움이 된다.
3. 소켓 통신에 대해 잘못 설명한 것은?
 - ① 통신 소프트웨어는 서버 클라이언트로 나누어져 개발되어야 한다.
 - ② 소켓 주소는 IP 주소와 포트 주소로 구성된다.
 - ③ 소켓은 서버 소켓과 클라이언트 소켓의 두 종류가 있다.
 - ④ 서버 소켓은 서버 쪽에서 데이터를 전송하는데 사용되고 클라이언트 소켓은 클라이언트 쪽에서 데이터를 전송하는데 사용된다.
4. 통신 프로그램이 상대 통신 프로그램에 접속할 때 필요한 것은?

① IP 주소	② IP 주소와 포트 주소
③ 호스트 이름	④ 프로토콜과 호스트 이름
5. 다음 코드에 대한 설명으로 틀린 것은?

```
Socket socket = new Socket("192.168.0.115", 5050);
```

- ① 클라이언트 소켓 객체를 생성한다.
- ② 클라이언트가 접속할 서버의 주소는 192.168.0.115이다.
- ③ 생성된 클라이언트 소켓의 포트 번호는 5050이다.
- ④ 소켓 객체가 생성될 때 서버에 바로 접속한다.



6. 다음은 통신 서버 코드의 일부분이다. 주석에 따라 빈칸에 코드를 작성하라.

```
ServerSocket ss = _____ // 5550포트와 결합하는 서버 소켓 생성
Socket s = _____ // 클라이언트로부터의 접속을 기다린다.
BufferedWriter out = new BufferedWriter(new
    OutputStreamWriter(s._____); // 소켓 s로 데이터를 전송할 출력 스트림을 만든다.
out._____ // 소켓으로 하여금 "안녕"을 전송하도록 한다.
s._____ // 소켓 s를 닫는다.
_____ // 서버 소켓을 닫는다.
```

7. ServerSocket에 대한 다음 설명으로 틀린 것은?

```
ServerSocket ss = new ServerSocket(5050);
```

- ① 서버 컴퓨터에서 서버 소켓을 생성하는 코드이다.
- ② 서버 소켓이 클라이언트로부터 접속을 받는 포트 번호는 5050이다.
- ③ 생성된 ss 소켓은 서버에서 클라이언트와 데이터를 주고받기 위해 사용된다.
- ④ ServerSocket은 Socket 클래스에는 없는 accept() 메소드를 가지고 있다.

8. 다음은 통신 클라이언트 코드의 일부분이다. 주석에 따라 빈칸에 코드를 작성하라.

```
Socket s = _____ // 203.1.1.1주소의 6000번 포트에 접속을 시도한다.
BufferedReader in = new BufferedReader(new
    InputStreamReader(s._____); // 소켓 s로부터 데이터를 수신할 입력 스트림을 만든다.
in._____ // 소켓으로부터 한 라인의 텍스트를 입력받는다.
s._____ // 소켓 s를 닫는다.
```

실습문제

• 홀수 문제는 정답이 공개됩니다.

1. 클라이언트에서는 한 줄씩 텍스트를 입력 받아 서버로 보내고, 서버는 받은 텍스트를 출력하는 소켓 프로그램을 작성하라. 클라이언트가 "끝"을 전송하면 클라이언트와 서버 모두 연결을 끊고 종료하라. 실행 예시는 다음과 같다. **난이도 6**

목적 클라이언트에서는 보내고 서버에는 받기만 하는 1:1 통신 만들기

서버입니다. 클라이언트를 기다립니다...
연결되었습니다.
... 자바 프로그램을 짜고 있어.
... 굉장히 재밌어.
... 소켓 프로그램이 더 그래
... 다했다. 이제 자야겠어
접속을 종료합니다.

서버 화면

←
접속,
텍스트
전송

서버에 접속하였습니다...
텍스트 입력 >> 자바 프로그램을 짜고 있어.
텍스트 입력 >> 굉장히 재밌어.
텍스트 입력 >> 소켓 프로그램이 더 그래
텍스트 입력 >> 다했다. 이제 자야겠어
텍스트 입력 >> 끝
연결을 종료합니다.

클라이언트 화면

2. 서버는 클라이언트가 접속하자마자 500밀리초마다 클라이언트에게 1부터 시작하여 1씩 증가하는 숫자를 전송하여 시간을 알려주는 소켓 통신 프로그램을 작성하라. 서버는 10초가 지나면 연결을 끊고 종료한다. **난이도 6**

목적 서버는 보내기만 하고 클라이언트는 받기만 하는 1:1 통신 만들기

타임서버입니다.
연결되었습니다.
500ms마다 수를 보냅니다.
종료합니다.

서버 화면

←
접속
→
숫자
전송

서버에 접속하였습니다...
서버의 시간 : 0 1 2 3 4 5 6 7 8 9 10
11 12 13 14 15 16 17 18 19
연결을 종료합니다.

클라이언트 화면



서버에서는 500ms마다 수를 보내는 `TimerThread`를 만들고 `main()`에서는 다음과 같이 스레드 객체를 생성한다. `TimerThread`는 `run()`에서 20번 수를 보낸 후 종료한다.

```
out = new BufferedWriter(new OutputStreamWriter(socket.getOutputStream()));
TimerThread th = new TimerThread(out); // 스레드 생성자에 스트림 out 전달
th.start();
try {
    th.join(); // 스레드 th가 종료하기를 기다린다.
} catch (InterruptedException e) { }
```

목적 다중 클라이언트를 서비스 하는 소켓 통신 응용

3. 하나의 서버에 다중 클라이언트가 동시에 접속하여 통신하는 프로그램을 작성해보자. 본문의 15.4의 코드를 스윙을 이용하여 실행 예시와 같이 수정 작성하라. 계산 서버에는 JTextArea 창을 두고 클라이언트가 접속했음과 클라이언트가 보내온 수식, 그리고 계산 결과를 출력하여 서버 사용자에게 알리고, 클라이언트는 계산 버튼을 누르면 계산식을 서버로 전송하고 결과를 받아 JTextField 결과 창에 출력한다. 난이도 7

현재 2명의 클라이언트가 접속되어 있음

JTextArea

```
멀티스레드 계산 서버
클라이언트 연결됨
20-55=35
클라이언트 연결됨
35*6=210
80@4=잘못된 연산
```

서버

```
계산 클라이언트
20 - 55 = -35 계산
```

클라이언트 1

계산 버튼을 눌러 서버로부터 받은 계산값

```
계산 클라이언트
80 @ 4 = 잘못된 연산 계산
```

클라이언트 2

목적 다중 클라이언트를 서비스 하는 소켓 통신 응용

4. 하나의 서버에 다중 클라이언트가 동시에 접속하여 통신하는 프로그램을 작성해보자. 서버는 영어 단어 25143개를 가지고 있는 words.txt 파일을 읽고, 클라이언트는 사용자로부터 영어 단어를 입력받아 서버로 보낸다. 서버는 클라이언트로부터 받은 단어가 words.txt에 있는지 검사하고 있으면 "YES", 없으면 "NO"를 전송한다. 클라이언트는 서버로부터 받은 결과를 출력한다. words.txt 파일은 독자들에게 배포하는 자료에 포함되어 있다. 난이도 7

2명의 클라이언트가 접속되어 있음

```
영어 스펠 체크 서버
words.txt 읽기 완료
클라이언트 연결됨
kite=YES
클라이언트 연결됨
friend=YES
pather=NO
```

서버

```
스펠체크 클라이언트
단어 입력 kite kite는 YES
```

클라이언트 1

kite 단어가 있음

<Enter> 키를 입력하면 서버로 단어 전송

```
스펠체크 클라이언트
단어 입력 pather pather는 NO
```

클라이언트 2

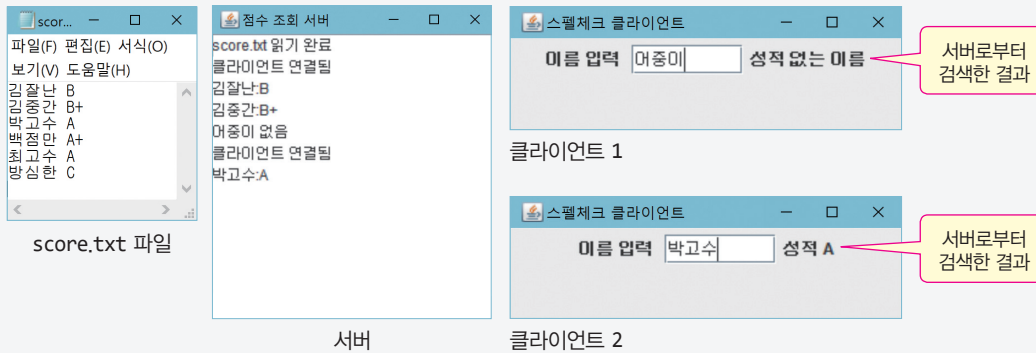
pather 단어는 없음



words.txt 파일을 읽어 Vector<String> 컬렉션에 저장하라. 8장 open challenge 참고.

5. 학생 이름과 학점이 저장된 텍스트 파일을 미리 `c:\temp\score.txt`에 마련해두고, 서버는 이 파일을 읽는다. 그리고 클라이언트로부터 학생 이름을 입력받으면 학점을 전송하는 통신 프로그램을 작성하라. 여러 클라이언트가 서버에 접속할 수 있도록 하라. **난이도 8**

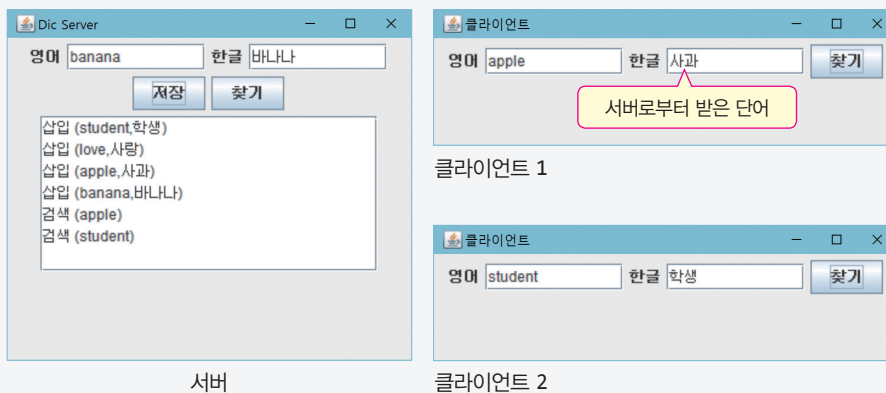
목적 서버의 파일을 검색하는
다중 클라이언트 서비스

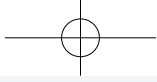


score.txt 파일을 읽어 `HashMap<String, String>` 컬렉션에 저장하라.

6. 서버의 관리자는 서버 프로그램의 화면에서 영한사전의 데이터를 저장할 수 있고, 클라이언트는 서버에 영어 단어를 보내 한글 단어를 검색할 수 있다. 다음은 실행 화면으로 서버에서 (student, 학생), (love, 사랑), (apple, 사과), (banana, 바나나)의 4 단어를 입력하였다. 그리고 클라이언트 프로그램을 실행하여 love 단어와 apple을 검색하는 화면이다. 사용자가 클라이언트에서 "apple"을 입력하고 "찾기" 버튼을 누르면 클라이언트는 서버에 "apple"을 보낸다. 서버는 현재 입력된 단어 중에서 "apple"을 찾고 "사과"를 전송한다. 클라이언트는 받은 "사과"를 출력한다. **난이도 8**

목적 소켓 통신, HashMap 등의
복합적인 응용프로그램 작성 연습





📌 소켓 통신으로 파일 전송
응용 만들기 연습. 바이너리 데
이터 전송

7. GUI 없이 콘솔에서 실행되는 파일 전송 프로그램을 작성해보자. 클라이언트에서 서버로 바이너리 파일을 전송하여 서버에 똑같은 사본을 저장하는 프로그램을 작성하라. 클라이언트와 서버는 다음과 같은 간단한 프로토콜을 이용하라. 편의상 클라이언트는 서버에게 명령과 인자를 오류 없이 전송하는 것으로 간주하고 오류 복구를 위한 재전송은 없는 것으로 한다. 난이도 8

명령(1바이트)	명령 인자
FILE_NAME(0x00)	첫 4바이트(파일 이름 바이트 크기), 파일 이름
FILE_SIZE(0x01)	파일 크기(8바이트)
SEND_BEGIN(0x02)	인자 없음
SEND_END(0x03)	인자 없음

```
연결됨
전송받은 파일 이름 길이 :8
전송받은 파일 이름:back.jpg
저장할 파일 이름:copy_back.jpg
전송받은 파일 크기:561276
.....
.....
.....
.....
파일 수신 성공. 현재 폴더에 저장되었습니다.
```

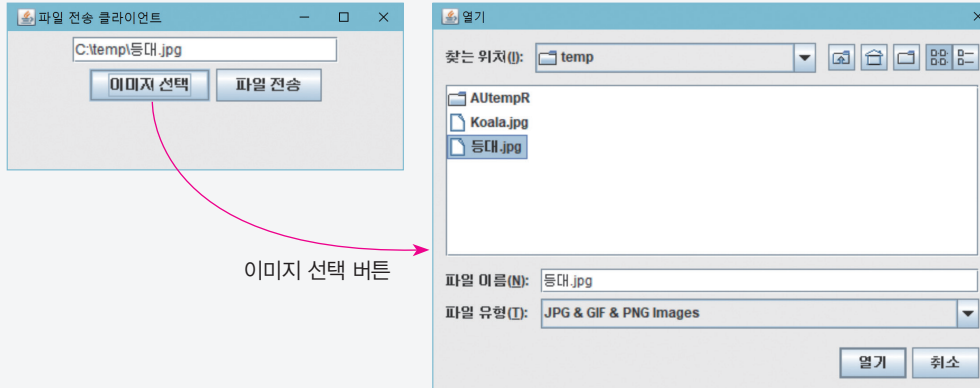
서버

```
보낼 파일 이름을 입력하세요>>c:\temp\
back.jpg
파일 전송이 완료되었습니다.
```

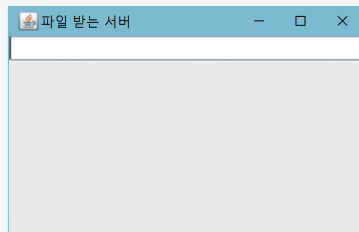
클라이언트

8. 문제 7의 프로그램을 스윙을 이용하여 재작성하라. 서버를 먼저 실행시키고, 클라이언트에서 '이미지 선택' 버튼을 누르면 JFileChooser를 이용하여 사용자로부터 이미지 파일을 선택받는다. 그리고 '파일 전송' 버튼을 누르면 이미지 파일을 서버로 전송한다. 서버는 받은 이미지를 화면에 출력한다. **난이도 9**

목적 스윙으로 바이너리 파일 전송 프로그램 작성. 종합 응용

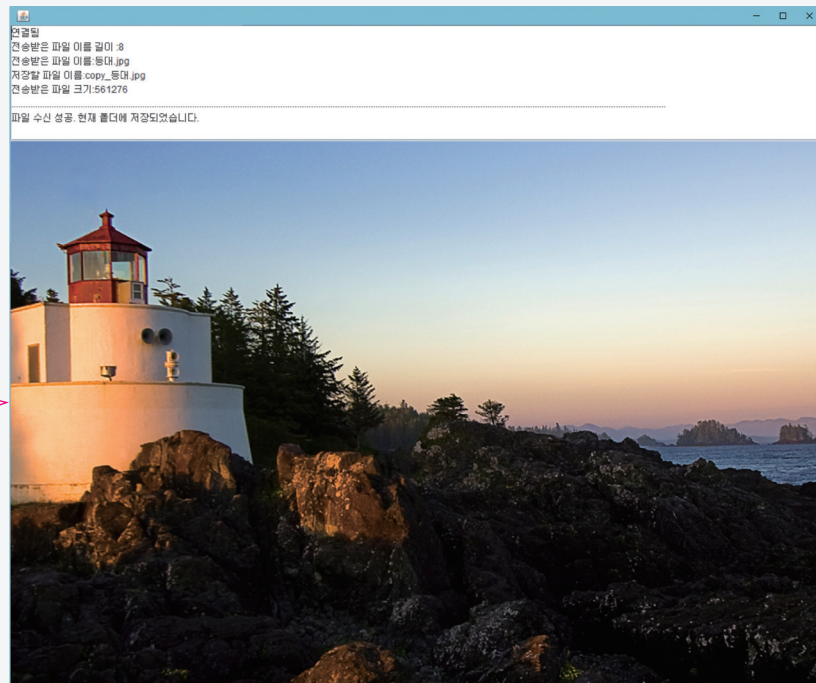


이미지 선택 버튼

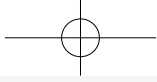


초기 서버 화면

클라이언트로부터 이미지 파일을 받으면 서버에 출력된다.



이미지를 받은 후 서버 화면



15장 Check Time 정답

Check Time(p.827)

1. IP 주소는 네트워크상에서 유일하게 식별될 수 있는 네트워크 장치의 주소이며, 한 컴퓨터 내의 각 응용프로그램은 통신을 위해 가상의 연결단인 포트를 생성하고 이 포트 번호로 자신을 식별하게 한다.
2. 윈도우 PC에서 명령창을 열어 `ipconfig` 명령을 수행한다.
3. 컴퓨터가 데이터를 수신하게 되었을 때 어떤 프로그램에게 전달해 주어야 할지 판단할 수 없게 된다.
4. 하나의 통신 프로그램은 여러 개의 포트를 사용할 수 있다.